

CSE 3204
Formal Language, Automata and Computability



Lecture 26
Restricted Satisfiability Problem

Normal Forms for Boolean Expressions

The following are three essential definitions:

- A *literal* is either a variable, or a negated variable. Examples are x and $\neg y$. To save space, we shall often use an overbar $\bar{y}$ in place of a literal such as $\neg y$.
- A *clause* is the logical OR of one or more literals. Examples are x , $x \vee y$, and $x \vee \bar{y} \vee z$.
- A boolean expression is said to be in *conjunctive normal form*³ or *CNF*, if it is the AND of clauses.

To further compress the expressions we write, we shall adopt the alternative notation in which $\vee$ is treated as a sum, using the $+$ operator, and $\wedge$ is treated as a product. For products, we normally use juxtaposition, i.e., no operator, just as we do for concatenation in regular expressions. It is also then natural to refer to a clause as a “sum of literals” and a CNF expression as a “product of clauses.”

$$(x \vee \neg y) \wedge (\neg x \vee z) \longrightarrow \overline{(x + y)}(\bar{x} + z)$$

$$(x + y\bar{z})(x + y + z)(\bar{y} + \bar{z}) \quad \text{is not CNF.}$$

$$xyz \text{ is in CNF}$$

An expression is said to be in *k-conjunctive normal form* (*k*-CNF) if it is the product of clauses, each of which is the sum of exactly k distinct literals. For instance, $(x + \bar{y})(y + \bar{z})(z + \bar{x})$ is in 2-CNF, because each of its clauses has exactly two literals.

- CSAT is the problem: given a boolean expression in CNF, is it satisfiable?
- k SAT is the problem: given a boolean expression in k -CNF, is it satisfiable?

We shall see that CSAT, 3SAT, and k SAT for all k higher than 3 are NP-complete. However, there are linear-time algorithms for 1SAT and 2SAT.

Converting Expressions to CNF

- Two Boolean expressions are equivalent if they have the same result on any truth assignment to their variables.
- If two expressions are equivalent, then surely either both are satisfiable or neither is.
- converting arbitrary expressions to equivalent CNF expressions is a promising approach to developing a polynomial time reduction from SAT to CSAT. However, it may take exponential time.
- Take a SAT instance E and convert it to a CSAT instance F such that F is satisfiable if and only if E is.
- It is not necessary that E and F be equivalent, have the same set of variables. In fact, generally F will have a superset of the variables of E .

Converting Expressions to CNF

- We need to convert an expression **E** to CNF **F** such that **F** is satisfiable if and only if **E** is satisfiable.

The reduction of SAT to CSAT will consist of two parts. First, we push all $\neg$'s down the expression tree so that the only negations are of variables; i.e., the boolean expression becomes an AND and OR of literals. This transformation produces an equivalent expression and takes time that is at most quadratic in the size of the expression. On a conventional computer, with a carefully designed data structure, it takes only linear time.

The second step is to write an expression that is the AND and OR of literals as a product of clauses; i.e., to put it in CNF. By introducing new variables, we are able to perform this transformation in time that is a polynomial in the size of the given expression. The new expression F will not be equivalent to the old expression E , in general. However, F will be satisfiable if and only if E is. More specifically, if T is a truth assignment that makes E true, then there

Our first step is to push $\neg$'s below $\wedge$'s and $\vee$'s. The rules we need are:

1. $\neg(E \wedge F) \Rightarrow \neg(E) \vee \neg(F)$. This rule, one of *DeMorgan's laws*, allows us to push $\neg$ below $\wedge$. Note that as a side-effect, the $\wedge$ is changed to an $\vee$.
2. $\neg(E \vee F) \Rightarrow \neg(E) \wedge \neg(F)$. The other “DeMorgan's law” pushes $\neg$ below $\vee$. The $\vee$ is changed to $\wedge$ as a side-effect.
3. $\neg(\neg(E)) \Rightarrow E$. This *law of double negation* cancels a pair of $\neg$'s that apply to the same expression.

is an *extension* of T , say S , that makes F true; we say S is an extension of T if S assigns the same value as T to each variable that T assigns, but S may also assign a value to variables that T does not mention.

Converting Expressions to CNF

Example 10.11: Consider the expression $E = \neg\left(\left(\neg(x + y)\right)(\overline{x} + y)\right)$. Notice that we have used a mixture of our two notations, with the $\neg$ operator used explicitly when the expression to be negated is more than a single variable. Figure 10.6 shows the steps in which expression E has all its $\neg$'s pushed down until they become parts of literals.

The final expression is equivalent to the original and is an OR-and-AND expression of literals. It may be further simplified to the expression $x + y$, but that simplification is not essential to our claim that every expression can be rewritten so the $\neg$'s appear only in literals. $\square$

Expression	Rule
$\neg\left(\left(\neg(x + y)\right)(\overline{x} + y)\right)$	start
$\neg\left(\neg(x + y)\right) + \neg(\overline{x} + y)$	(1)
$x + y + \neg(\overline{x} + y)$	(3)
$x + y + \left(\neg(\overline{x})\right)\overline{y}$	(2)
$x + y + x\overline{y}$	(3)

Converting Expressions to CNF

Theorem 10.12: Every boolean expression E is equivalent to an expression F in which the only negations occur in literals; i.e., they apply directly to variables. Moreover, the length of F is linear in the number of symbols of E , and F can be constructed from E in polynomial time.

Proof from the book.

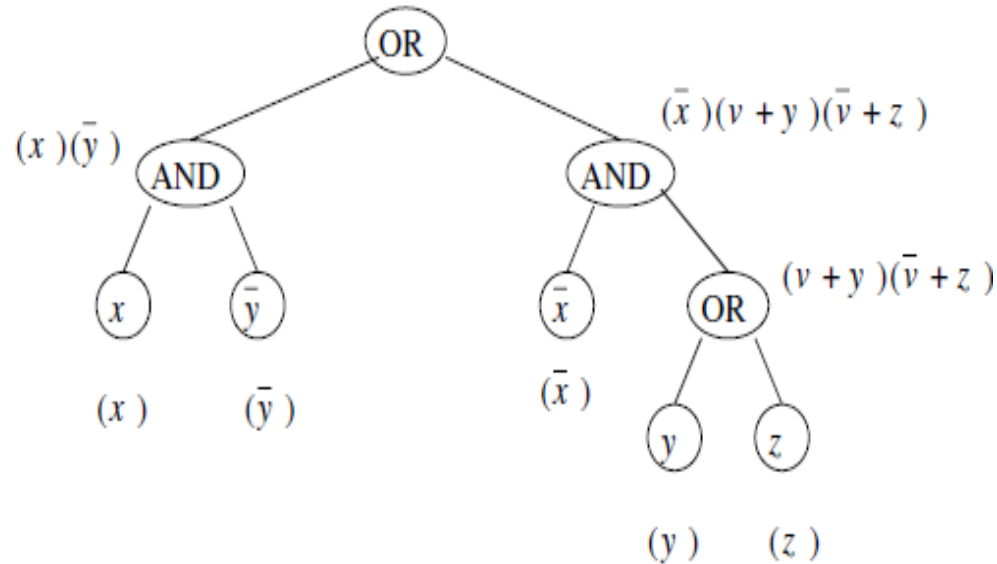
NP Completeness of CSAT.

- Proof from the book.

Converting Expressions to CNF

$$E = x\bar{y} + \bar{x}(y + z)$$

$$(u + x)(u + \bar{y})(\bar{u} + \bar{x})(\bar{u} + v + y)(\bar{u} + \bar{v} + z)$$



$T(x) = 0, T(y) = 1$, and $T(z) = 1$ satisfies E . We can extend T to S by adding $S(u) = 1$ and $S(v) = 0$ to the required $S(x) = 0, S(y) = 1$, and $S(z) = 1$ that we get from T . You may check that S satisfies F .

$S(u)=1$ is chosen to make the first part of E satisfiable.

Figure 10.7: Transforming a boolean expression into CNF

NP-Completeness of 3SAT Problem

- Given a boolean expression E that is the product of clauses, each of which is the sum of three distinct literals, is E satisfiable?

PROOF: Evidently 3SAT is in $\mathcal{NP}$, since SAT is in $\mathcal{NP}$. To prove NP-completeness, we shall reduce CSAT to 3SAT. The reduction is as follows. Given a CNF expression $E = e_1 \wedge e_2 \wedge \cdots \wedge e_k$, we replace each clause e_i as follows, to create a new expression F . The time taken to construct F is linear in the length of E , and we shall see that a truth assignment satisfies E if and only if it can be extended to a satisfying truth assignment for F .

- If e_i is a single literal, say (x) ,⁵ introduce two new variables u and v . Replace (x) by the four clauses $(x+u+v)(x+u+\bar{v})(x+\bar{u}+v)(x+\bar{u}+\bar{v})$. Since u and v appear in all combinations, the only way to satisfy all four clauses is to make x true. Thus, all and only the satisfying assignments for E can be extended to a satisfying assignment for F .

The construction of F requires time linear to the length of E .

- Suppose e_i is the sum of two literals, $(x+y)$. Introduce a new variable z , and replace e_i by the product of two clauses $(x+y+z)(x+y+\bar{z})$. As in case 1, the only way to satisfy both clauses is to satisfy $(x+y)$.
- If e_i is the sum of three literals, it is already in the form required for 3-CNF, so we leave e_i in the expression F being constructed.
- Suppose $e_i = (x_1+x_2+\cdots+x_m)$ for some $m \geq 4$. Introduce new variables $y_1, y_2, \dots, y_{m-3}$ and replace e_i by the product of clauses

$$\begin{aligned} & (x_1+x_2+y_1)(x_3+\bar{y}_1+y_2)(x_4+\bar{y}_2+y_3)\cdots \\ & (x_{m-2}+\bar{y}_{m-4}+y_{m-3})(x_{m-1}+x_m+\bar{y}_{m-3}) \end{aligned} \quad (10.2)$$

An assignment T that satisfies E must make at least one literal of e_i true; say it makes x_j true (recall x_j could be a variable or a negated variable). Then, if we make $y_1, y_2, \dots, y_{j-2}$ true and make $y_{j-1}, y_j, \dots, y_{m-3}$ false, we satisfy all the clauses of (10.2). Thus, T may be extended to satisfy these clauses. Conversely, if T makes all the x 's false, it is not possible to extend T to make (10.2) true. The reason is that there are $m-2$ clauses, and each of the $m-3$ y 's can only make one clause true, regardless of whether it is true or false.

- Section 10.3 of Chapter 10.